

Project Interim Report
On
Student Grading System
(Using Core Java & OOP Concepts)

RU-100-01-00012
Object Oriented Programming with Java

SESSION: 2025-26



Submitted By
AYUSH KUMAR
RU-25-11723

**Bachelor of Technology in Computer Science &
Engineering with AI in association with GOOGLE**

Under the Guidance of
ASHISH SHIVHARE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

The Student Grading System is a Java-based application developed using Object-Oriented Programming (OOP) concepts. The purpose of this project is to manage student academic records efficiently by storing student details such as name and marks, calculating the average score, and assigning grades based on predefined criteria.

In traditional systems, grading is often done manually, which can lead to errors and inefficiency. This project helps automate the grading process, ensuring accuracy and saving time. The system takes student marks as input, computes the average, and assigns grades automatically based on predefined grading criteria.

The project demonstrates key OOP concepts such as classes, objects, encapsulation, and methods. It provides a simple yet effective solution for academic evaluation and can be further enhanced by integrating databases and graphical user interfaces.

Introduction

In educational institutions, managing student performance is an essential task. Manual grading systems are time-consuming and prone to mistakes. To overcome this, the Student Grading System is designed using Java and OOP principles.

This system allows users to:

- Store student name and marks
- Calculate average marks
- Assign grades automatically
- Display complete student details

The project is built using a class-based approach where each student is treated as an object. This improves code structure, readability, and reusability.

Objectives

The main objectives of this project are:

- Store student records efficiently
- Calculate average marks automatically
- Assign grades based on performance
- Reduce manual calculation errors
- Implement OOP concepts practically

Scope

This project is suitable for:

- Schools and colleges for basic grading systems
- Beginners learning Java and OOP
- Academic projects demonstrating Object-Oriented Programming concepts

System Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4GB RAM

Software Requirements

- Java JDK
- IDE (VS Code / Eclipse / IntelliJ IDEA)

Methodology

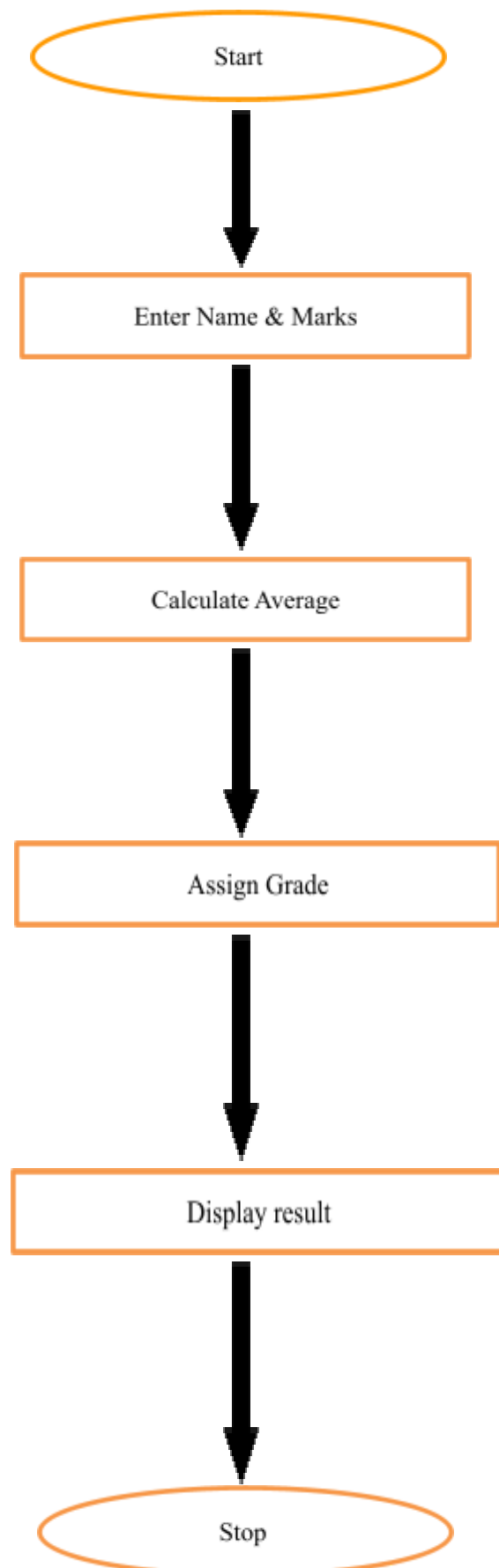
The working of the system follows these steps:

1. Create the Student class.
2. Accept student name and marks as input.
3. Store the entered details in class variables.
4. Calculate the average marks.
5. Assign a grade based on the average.
6. Display the student report.

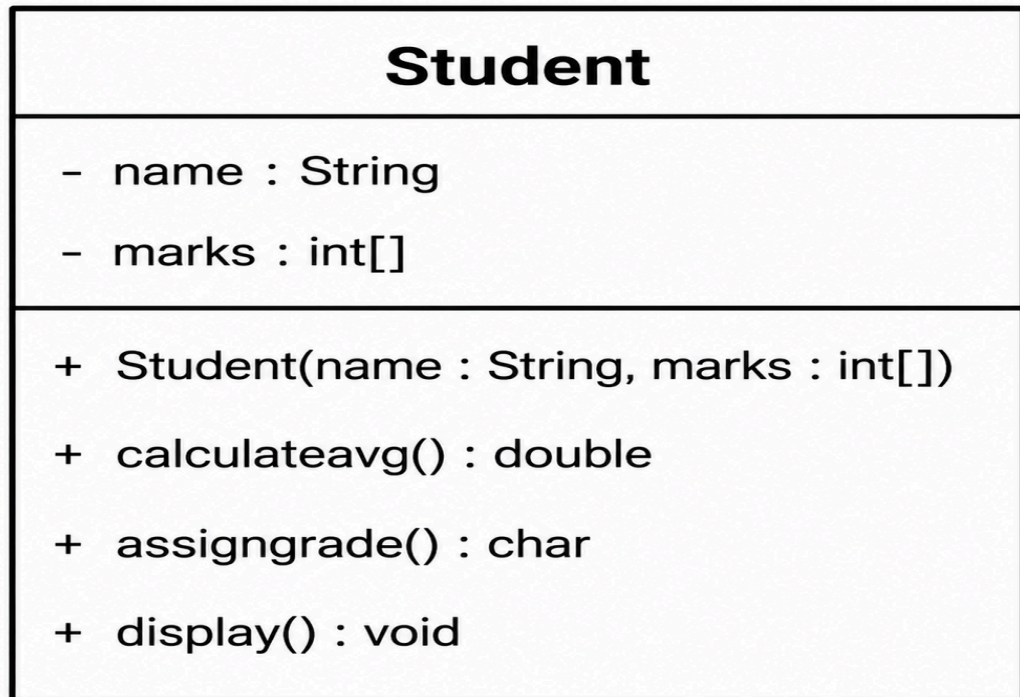
Flow of Execution

Start → Enter Name & Marks → Calculate Average → Assign Grade → Display Result → Stop

Flowchart



Class diagram



Implementation

This project uses one main class:

1. Student Class

- Stores student name and marks
- Stores multiple subject marks
- calculateavg() method calculates the average marks
- assigngrade() method assigns grade using if-else conditions
- display() method displays the student report

2. Main Class

- Accepts input using Scanner
- Creates an object of the Student class
- Invokes class methods
- Displays the final result

Sample Input / Output

Input:

Enter Student Name: Ayush kumar
Enter Number of Subjects: 2
Enter Marks:
Subject 1: 90
Subject 2: 95

Output:

STUDENT REPORT
Student Name : Ayush kumar
Marks : 90 95
Average : 92.5
Grade : A

Gantt Chart

Task	Duration
Planning	1 Day
Design	1 Day
Coding	2 Days
Testing	1 Day
Report Writing	1 Day

Conclusion

The Student Grading System project successfully demonstrates the use of Object-Oriented Programming concepts in Java. It simplifies the grading process by automating calculations and grade assignment. The project improves efficiency, reduces errors, and provides a structured approach to managing student data

References

1. Book

[1] H. Schildt, *Java: The Complete Reference*, 11th ed., McGraw-Hill, 2018.

2. Website

[2] Oracle, "Java Documentation." [Online]. Available: <https://docs.oracle.com>